

Phase 3 — Architectural Design Report

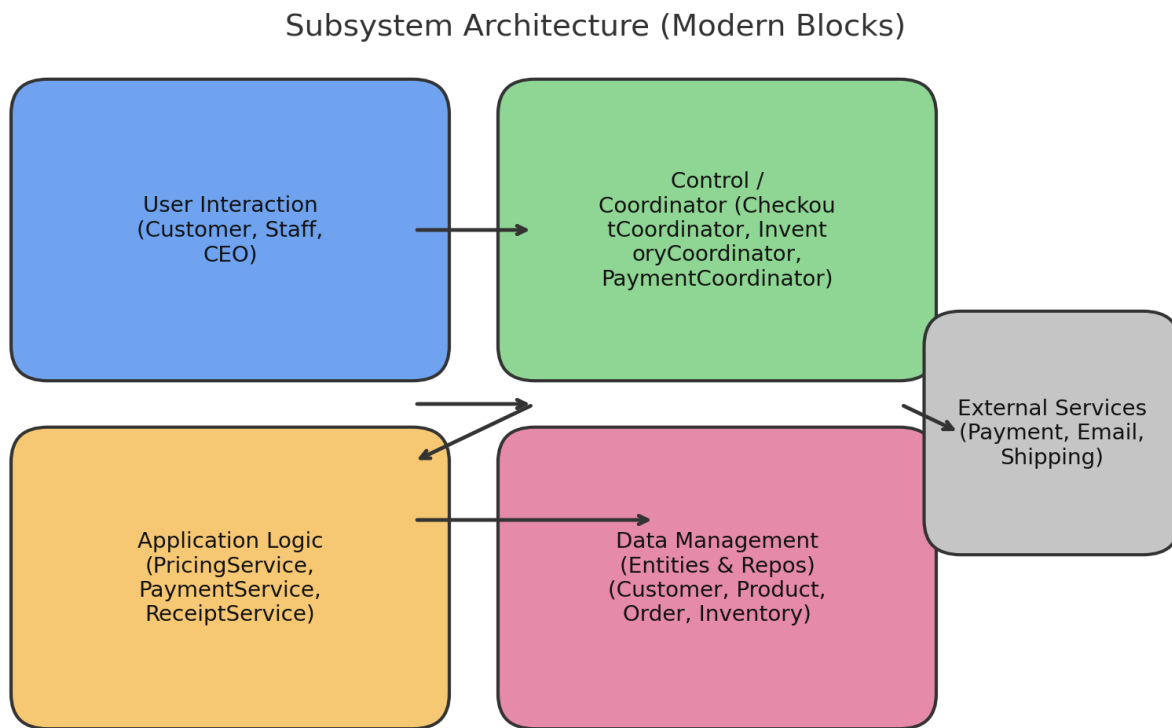
Generated: 2025-10-14 05:37 UTC

1. Introduction

This Phase 3 report presents the architectural design for the Shopping Mall App. It transitions the system from detailed analysis and static models (Phase 1 & 2) into a component and subsystem architecture, integrated communication diagram, and coordinator responsibilities. The design favors modularity, clear separation of concerns, and robustness for production-like flows.

2. Subsystem Design

The system is organized into four main subsystems: User Interaction, Control/Coordinator, Application Logic, and Data Management. See the subsystem diagram below.



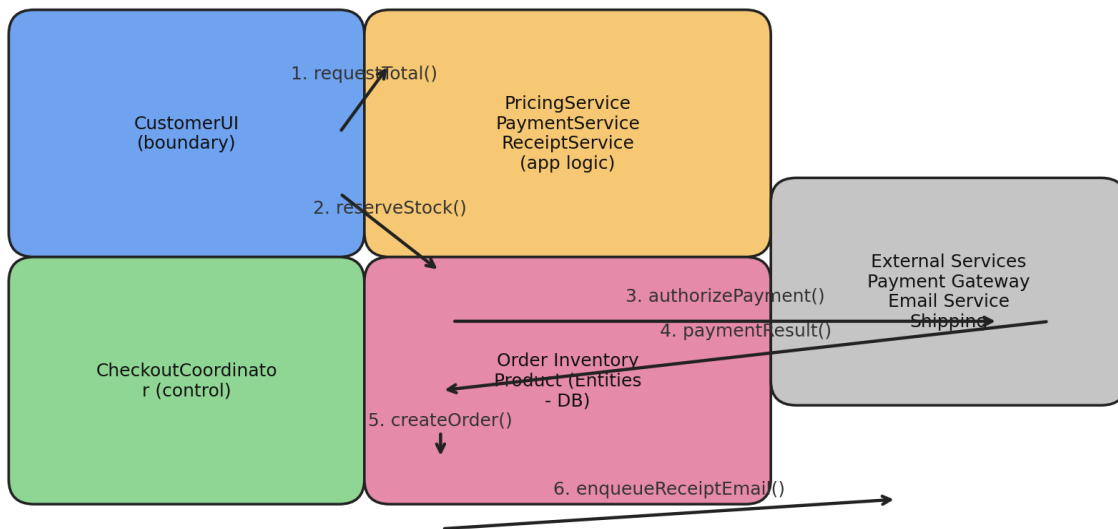
3. Communication Styles

- UI → Control: synchronous request/response (user-driven).
- Control → Application Logic: synchronous calls for business operations (calculate total, validate promos).
- Control → External Services: asynchronous patterns where appropriate (email queue, payment callbacks).
- Control ↔ Control: synchronous or orchestrated saga steps for multi-step flows.

4. Integrated Communication Diagram

This integrated, simplified communication diagram collapses duplicate boundary interfaces and shows the coordinator-based orchestration for Checkout and related services. See the figure below:

Integrated Communication (Simplified) - Modern Blocks



5. Coordinator Descriptions

CheckoutCoordinator

Orchestrates checkout: compute total (PricingService), reserve stock (InventoryCoordinator), authorize payment (PaymentCoordinator), create Order, commit reservation, issue Receipt, enqueue email. Persists minimal reservation/saga state for recovery.

InventoryCoordinator

Manages stock reservations, commits, releases, and low-stock alerts. Persists reservation tokens and inventory transaction logs.

PaymentCoordinator

Handles payment authorization, capture and refunds. Implements retry/backoff and saves transaction references for audit.

EmailCoordinator

Queues outgoing emails (order confirmations, receipts) and manages retry and dead-letter logic via EmailQueueItem and a worker.

ReportCoordinator

Schedules and runs reports (daily/monthly) via ReportService and persists generated reports as XLSX with fileRef.

OrderCoordinator

Manages order lifecycle transitions, cancellations, and manual reconciliations.

6. Prototype Implementation Plan (Suggested)

Language: Python (console prototype). Implement two demo flows:

- 1) Login + Registration: simple in-memory store for users, token-based session;
- 2) Checkout (simplified): add product to cart, compute total via PricingService, mock PaymentService to accept payments, and create an Order & Receipt (written to local file).

7. State & Persistence Notes

Key stateful components and required persisted fields include: Reservation tokens (InventoryCoordinator), Order.status, Payment records (status, transactionRef), EmailQueueItem (status, retryCount), Report.fileRef (XLSX). Use optimistic locking on Product.stockQty to avoid race conditions.

8. Conclusion

This Phase 3 document consolidates the architecture, clarifies responsibilities using coordinators, and resolves Phase 2 feedback by collapsing duplicate boundary interfaces and moving logic into application services and coordinators. The provided prototype plan is suitable for Phase 4 implementation.